

Rodrigo Fernandez

Senior Python (Django/FastAPI) Backend Engineer

Katy, Texas | +1 (407) 801-1287 | <https://www.linkedin.com/in/rodrigo-bermejo-fern%C3%A1ndez-9a5a0664/>
rodrigobfdev@gmail.com

Summary

Senior **Python Backend Engineer** with 12+ years of experience building secure, high-availability platforms across **cloud infrastructure, healthcare SaaS, fintech, eCommerce, and enterprise workflow industries**, specializing in **Python 3.8–3.12, Django 3.x–5.x, FastAPI 0.9x–0.11x, Flask 1.x–3.x, PostgreSQL 11–16, Redis 5–7, Celery 4–5, AWS, Docker, Kubernetes**, and event-driven APIs, with strong results migrating legacy services, fixing production defects, improving latency, hardening authentication, reducing cloud cost, and delivering backend systems that support reliable growth, audit readiness, and stable customer-facing operations.

Skills

- **Backend: Python 3.8–3.12, Django 3.x–5.x, Django REST Framework 3.x, FastAPI 0.9x–0.11x, Flask 1.x–3.x, SQLAlchemy 1.4–2.x, Pydantic 1.x–2.x, REST API, GraphQL, WebSocket, Microservices, Monolith Modularization**
- **Database: PostgreSQL 11–16, MySQL 5.7–8.x, MongoDB 4.x–6.x, Redis 5–7, DynamoDB, Elasticsearch 7–8, OpenSearch 1–2, Alembic, Database Indexing, Query Optimization**
- **Cloud & DevOps: AWS, EC2, ECS, Lambda, S3, RDS, CloudWatch, SQS, SNS, API Gateway, Docker, Kubernetes, Helm, Terraform, GitHub Actions, GitLab CI, Nginx**
- **Architecture: Event-Driven Architecture, Domain-Driven Design, Clean Architecture, CQRS, Background Jobs, Distributed Systems, Multi-Tenant SaaS, API Gateway, Authentication, Authorization**
- **Testing & Quality: Pytest, unittest, Factory Boy, Postman, Swagger/OpenAPI, Contract Testing, Integration Testing, Load Testing, Sentry, Datadog, Grafana**
- **Security: OAuth2, JWT, RBAC, SSO, IAM, Secrets Manager, OWASP, Rate Limiting, Data Encryption, Audit Logging**
- **Frontend Collaboration: React, TypeScript, JavaScript, HTML, CSS, API Integration, Admin Dashboard Support**

Experience

Lightedge — Senior Software Engineer *(Apr 2020 – Mar 2026)*

- Designed and maintained **Python 3.8–3.12** backend services for cloud infrastructure and enterprise hosting platforms, improving API stability and operational reliability for customer-facing workflows.
- Migrated critical Django services from **Django 3.x to Django 5.x** with dependency upgrades, middleware refactoring, and regression testing that reduced upgrade-related production issues by **35%**.
- Built high-throughput REST APIs using **FastAPI 0.9x–0.11x**, Pydantic validation, PostgreSQL, and Redis caching to support faster provisioning, billing, monitoring, and account management flows.
- Resolved recurring timeout bugs in reporting APIs by analyzing slow PostgreSQL execution plans, adding composite indexes, and reducing average query response time by **42%**.
- Improved background task reliability with **Celery 5.x**, Redis, retry policies, dead-letter handling, and idempotent task design for failed provisioning and notification workflows.
- Implemented role-based access control using **OAuth2**, JWT, permission scopes, and audit logging to strengthen compliance visibility across internal admin tools.

- Refactored legacy Flask endpoints into modular service layers, separating validation, business logic, and repository code to improve maintainability and reduce duplicated backend logic.
- Created AWS automation around **Lambda**, SQS, SNS, S3, and CloudWatch alarms to detect failed jobs earlier and reduce manual support investigation time by **30%**.
- Fixed memory leak issues in long-running Python workers by profiling object references, optimizing batch processing, and adding safer pagination for large customer datasets.
- Partnered with frontend engineers to define OpenAPI contracts, stabilize API response formats, and prevent breaking changes during React dashboard feature releases.
- Introduced Pytest integration suites, mocked external services, and CI checks that increased backend test coverage by **28%** while keeping release cycles flexible.
- Supported incident response by improving structured logs, Sentry traces, and dashboard metrics so engineers could isolate API, database, and queue failures faster.

NIX United — Software Engineer *(Oct 2016 – Mar 2020)*

- Developed **Python 3.6–3.10** backend systems for healthcare SaaS, fintech workflow, and marketplace clients using Django REST Framework, Flask, PostgreSQL, Redis, and AWS.
- Migrated several legacy APIs from **Flask 1.x** services into Django REST modules where stronger authentication, admin tooling, and serializer validation were required.
- Built secure patient and provider workflow APIs with **Django REST Framework 3.x**, JWT authentication, PostgreSQL transactions, and audit-friendly activity histories.
- Reduced invoice-processing API latency by **38%** by replacing inefficient ORM loops with optimized joins, select_related usage, and indexed lookup patterns.
- Implemented Celery-based background jobs for emails, PDF generation, webhook retries, and scheduled data synchronization between customer systems and internal services.
- Fixed production race conditions in payment status updates by introducing database row locking, idempotency keys, and safer retry handling for external gateway callbacks.
- Created reusable API modules for file upload, permissions, pagination, search filters, and notification preferences to speed up delivery across multiple client products.
- Integrated third-party services including Stripe, Twilio, SendGrid, and S3 while handling webhook verification, error recovery, and secure secret configuration.
- Improved deployment reliability using Docker, GitLab CI, environment-based settings, and migration checks that reduced failed staging deployments by **25%**.
- Worked flexibly across backend, API documentation, database design, and frontend integration support whenever delivery required stronger cross-team coordination.
- Added Pytest regression tests for critical authentication, billing, and data export flows, preventing repeated bugs during framework and dependency upgrades.

Webiz Digital — Software Developer *(Jan 2014 – Sep 2016)*

- Built **Python 2.7–3.7** and **Django 1.11–2.2** backend features for eCommerce, booking, CMS, and digital marketing platforms used by small and mid-sized businesses.
- Migrated older Django applications from function-based views to class-based views, cleaner service modules, and safer settings separation across development and production.
- Developed REST endpoints for product catalogs, order processing, customer profiles, coupons, and reporting dashboards using Django REST Framework and PostgreSQL.
- Improved checkout reliability by fixing session expiration bugs, payment callback mismatches, and duplicate order creation during unstable network retries.
- Optimized admin dashboard performance by adding database indexes, reducing unnecessary template queries, and improving page load speed by **33%** for large catalogs.

- Implemented asynchronous email, invoice, and image-processing jobs with Celery and Redis to keep customer-facing requests responsive during high-traffic periods.
- Integrated PayPal, Stripe, Mailchimp, Google Maps, and shipment tracking APIs while adding clear error handling for failed external service responses.
- Strengthened user authentication with password reset flows, permission checks, CSRF protection, and safer validation around account and order management forms.
- Supported frontend teams by shaping JSON contracts, troubleshooting browser API errors, and adjusting backend responses for smoother JavaScript integration.
- Created deployment scripts, environment configuration patterns, and rollback notes that made releases more predictable for client maintenance projects.

Aurio — Junior Software Developer *(Oct 2012 – Dec 2013)*

- Supported **Python 2.7–3.4** web applications using Django, Flask, MySQL, jQuery, HTML, CSS, and early REST patterns for business websites and internal tools.
- Built CRUD modules for customer records, staff workflows, inventory updates, and reporting pages while learning clean backend structure and reusable model design.
- Fixed form validation bugs, broken redirects, encoding issues, and database constraint errors that affected daily operations for internal business users.
- Improved legacy SQL queries by adding safer filters, reducing repeated database calls, and making report pages more reliable for larger datasets.
- Assisted with migration from older procedural PHP-style workflows into Python-based Django modules where maintainability and admin control were stronger.
- Created simple REST-style endpoints for JavaScript-driven pages, helping frontend screens update records without forcing full page reloads.
- Added logging around failed imports, email delivery problems, and user-submitted data errors so recurring support issues became easier to diagnose.
- Prepared basic unit tests for models, forms, and utility functions, helping senior engineers catch regressions before client demos and maintenance releases.
- Worked across backend fixes, small frontend adjustments, database updates, and deployment support to build flexible full-cycle engineering habits early.

Microsoft — Software Developer Intern *(Jun 2012 – Sep 2012)*

- Supported internal engineering tools using **C#, .NET Framework 4.0–4.5, ASP.NET MVC 3–4**, JavaScript, jQuery, and SQL Server, gaining practical experience in enterprise software quality and structured development workflows.
- Assisted senior engineers with debugging UI defects, validation issues, and data-handling problems, documenting root causes clearly so fixes could be reviewed and released with lower regression risk.
- Built small internal web modules for tracking records, reviewing status changes, and improving team visibility, applying clean HTML, CSS, JavaScript, and backend integration patterns.
- Resolved early-stage bugs related to inconsistent form validation and SQL query results by reproducing issues locally, checking logs, and working with mentors to apply safer input-handling rules.

Education

Texas State University

Bachelor's Degree in Computer Science *(Sep 2008 – May 2012)*